FinPal

Financial Agent Buddy - Product Requirements Document

PayPal AI Hackathon | Deadline: 12 November 2026

Team: Dave - Henry (Frontend) - Celestine & Steph (Backend)

1. Product Overview

FinPal is an agentic financial assistant built on PayPal Sandbox. It turns user-created purchase proposals and calendar context into constrained, safety-checked spending. It checks permission, purchase value, and merchant trust, then leaves an auditable decision record.

Positioning: An AI agent that spends within your rules, verifies value using market data, checks the merchant, and leaves a record that can be reviewed or reversed through the correct PayPal operation.

2. Goals and Scope

The hackathon goal is a polished end-to-end prototype: proposal, constraints, deterministic quality score, decision, PayPal authorization/capture, and audit history.

- Must: PayPal Sandbox, Channel3, AG Grid, constraints, decision orchestration, and append-only audit events.
- Should: Bryntum Calendar and a basic KERNEL merchant trust check.
- Extra only after the core loop works: Elastic, Astropods, Postman automation, Zapier, and additional timelines.
- Sandbox-only: centrally configured merchant credentials; no user PayPal OAuth.

3. Locked Technical Decisions

Area	Decision	Implementation note
Application	Next.js App Router monorepo	Use Next.js route handlers/API routes; no FastAPI.
Authentication	Supabase Auth	Supabase also provides Postgres and per-user access control.
Frontend	React, Tailwind, shadcn/ui	AG Grid is the primary audit and transaction dashboard.
Payments	PayPal Sandbox	Central merchant credentials from environment variables only.
Deployment	Render backend/API	Frontend may use Vercel or Render; Render is the backend source of truth.
Calendar	Google Calendar plus Bryntum	Calendar events prefill proposals; they do not infer products.
Product data	Channel3	Deterministic scoring uses comparable offers.
Merchant trust	KERNEL basic check	A pass/fail/unavailable signal is sufficient for v1.

4. Core User Flow

- User signs in with Supabase Auth and configures time windows and amount caps.
- User manually creates a proposal or clicks Create proposal from event. Calendar data only prefills title and date.
- User supplies product name, proposed price, and currency.
- The API runs constraints, Channel3 Quality Score v1, and optional merchant trust checks.
- The decision matrix returns AUTO_APPROVE, MANUAL_REVIEW, or BLOCK with reasons.
- An approved decision creates a PayPal order and authorization. Capture occurs only after final approval.
- An AUTHORIZED payment can be VOIDED. A CAPTURED payment can only be REFUNDED if refunds are implemented.
- Every transition is appended to audit_events and displayed in AG Grid.

5. Quality Score v1

The score is deterministic. An LLM may explain the result but must not choose or change the category.

Condition	Score	Approval implication
At least 3 comparable offers and proposed price <= median x 1.05	Good	Eligible for auto-approval if all other checks pass.
At least 3 offers and proposed price <= median x 1.20	Fair	Manual review; merchant trust must pass.
At least 3 offers and proposed price > median x 1.20	Poor	Block.
Fewer than 3 comparable offers or unusable currency data	Insufficient data	Manual review; never auto-approve.

Inputs: proposed price, offer prices, and currency. Normalize or reject mismatched currencies before calculating the median. Store the category, explanation, offer count, median, lowest offer, currency, and Channel3 references used. Thresholds must also appear in code comments and tests.

6. Approval Decision Table v1

Constraints	Quality	Trust	Amount	Result
Pass	Good	Pass or N/A	Under cap	Auto-approve
Pass	Fair	Pass	Under cap	Manual review
Fail	Any	Any	Any	Block
Any	Poor	Any	Any	Block
Any	Any	Any	Over cap	Block
Any	Insufficient data	Any	Any	Manual review

Precedence: any explicit block condition wins. Otherwise insufficient data routes to manual review. Auto-approval requires every auto-approval condition to pass.

7. PayPal Payment State Model

From	Operation	To	Rule
-	Create order	CREATED	Persist PayPal order ID and idempotency key.
CREATED	Buyer/order approval	APPROVED	Record approval before authorization.
APPROVED	Authorize	AUTHORIZED	Funds are authorized but not captured.
AUTHORIZED	Capture	CAPTURED	Capture only after final approval.
AUTHORIZED	Void	VOIDED	Void is valid only before capture.
CAPTURED	Refund (optional)	REFUNDED	Never label this operation as void.

The UI, API payloads, payment_attempts table, and audit events must use these exact uppercase states.

8. Minimal Data Model

Table	Required fields / behavior
users	Supabase Auth identity/profile reference.
constraints	user_id, time_windows, amount_caps, currency, created_at, updated_at.
proposals	user_id, product_name, proposed_price, currency, source, source_event_id, proposed_for.
evaluations	proposal_id, quality_score, explanation, Channel3 references and prices, trust_signal.
decisions	evaluation_id, outcome, reason, decided_at, reviewer_id when manual.
payment_attempts	decision_id, PayPal order/authorization/capture IDs, status, amount, idempotency_key.
audit_events	append-only: user_id, entity_type, entity_id, type, payload, created_at.

9. Security and Operations

- Keep sandbox credentials and all third-party secrets in environment variables; never expose them to the browser or commit them.
- Require an idempotency key for payment creation and reject conflicting replays.
- Make audit_events insert-only for application roles; corrections are new events, not updates or deletes.
- Use Supabase row-level security so users can access only their own proposals, evaluations, decisions, and constraints.
- If PayPal webhooks are included, verify signatures and make handlers idempotent. Webhooks are optional for the demo.
- Store third-party request IDs and sanitized responses for troubleshooting; never log secrets.

10. Feature Priorities and Ownership

Priority	Feature	Owner
P0	Next.js/Supabase foundation and Auth	Frontend + Backend
P0	PayPal authorization, capture, and void	Backend
P0	Constraints and decision matrix	Backend
P0	Channel3 Quality Score v1	Backend
P0	Append-only audit log and AG Grid dashboard	Backend + Frontend
P1	Manual approval flow	Frontend + Backend
P1	Calendar proposal prefill and Bryntum view	Frontend + Backend
P1	Basic KERNEL merchant trust signal	Backend
P2	Refund, Elastic, Astropods, Zapier, extra timeline	Only if core is stable

11. Definition of Done

- A user signs in, creates constraints, and manually creates a proposal or prefills one from a calendar event.
- The proposal runs constraints and Quality Score v1, showing reference prices and an explanation.
- The decision matrix visibly produces auto-approve, manual review, or block.
- An approved sandbox order progresses through the exact PayPal states and can be voided while AUTHORIZED.
- All decisions and payment transitions appear in a filterable AG Grid audit dashboard.
- The public repository includes setup instructions, .env.example, schema/migrations, tests for scoring and decision rules, and sandbox-only disclosures.
- A sub-three-minute demo shows one successful purchase and at least one blocked or reviewed proposal.

12. Explicitly Out of Scope

- User PayPal OAuth, live payments, product inference from calendar text, multi-calendar support, behavioral duress detection, biometrics, crypto address masking, broad conversational BI, and complex multi-agent orchestration.

13. Success for Judges

Within three minutes, a judge should see that FinPal evaluates value and safety, enforces user-defined limits, uses PayPal authorization correctly, and preserves a transparent decision trail. Sponsor tools should support that loop rather than appear as disconnected extras.

Document version: 14 October 2026 - Locked architecture and v1 specifications